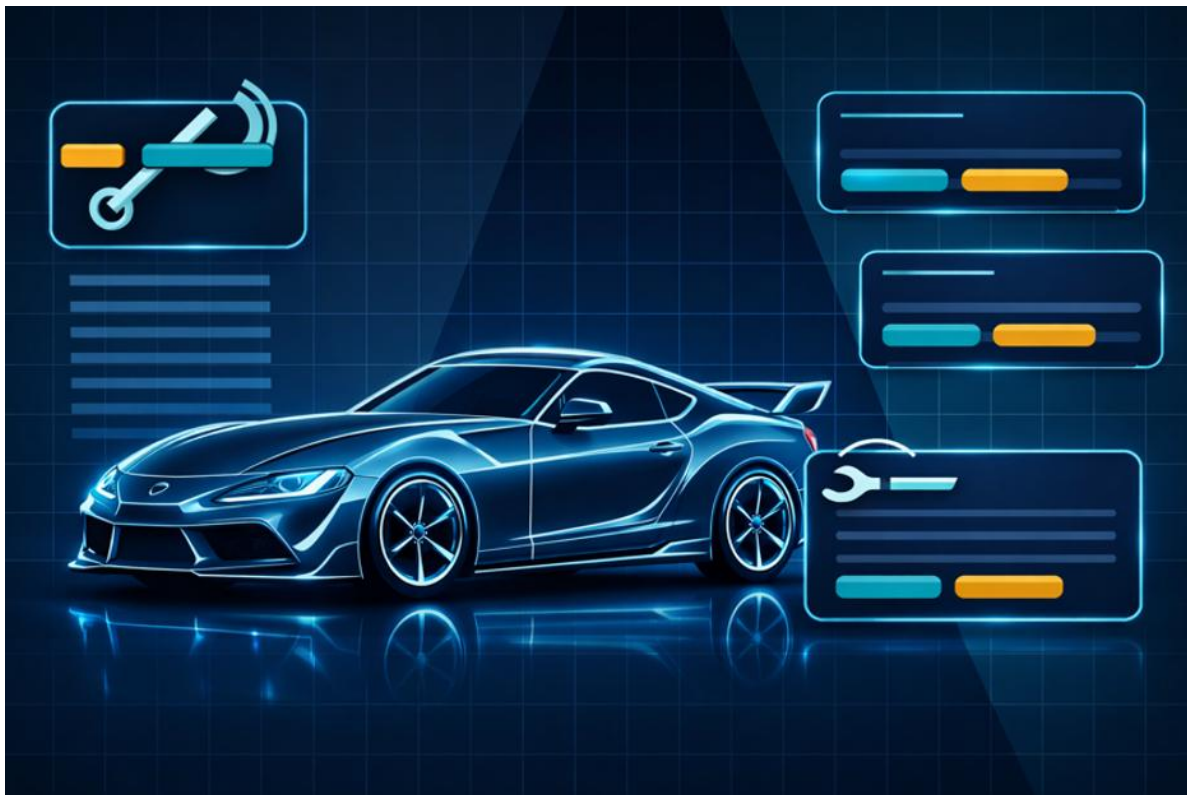


Juventus Technikerschule HF - Software Engineering 1

Projektarbeit

WerkstattPilot

Digitale Auftragsabwicklung für KFZ-Werkstätten



Titelbild WerkstattPilot

Angabe	Wert
Datum	10.06.2026
Autoren	Livio Luna, Dominic Lolos
Arbeitsgruppe	Arbeitsgruppe 3
Klasse	TS TSI 2502 A
Dozent	Arif Chughtai
Abgabeform	Projektdokument, Quelltext-ZIP, Präsentation und Arbeitsprotokoll

Inhaltsübersicht

Das Dokument fasst die fünf Teilaufgaben der Projektarbeit in einem einheitlichen Projektdokument zusammen. Jede Aufgabe ist als eigenes Hauptkapitel geführt. Tabellen, Diagramme und Nachweise sind direkt dem passenden Kapitel zugeordnet.

Kapitel	Titel	Inhalt
1	Aufgabe 1: Projektauftrag	Projektauftrag und priorisierte Anforderungen für WerkstattPilot
2	Aufgabe 2: Projektumgebung definieren und dokumentieren	Gemeinsame Projektumgebung, Ordnerstruktur und Entwicklungswerkzeuge
3	Aufgabe 3: Objektorientierte Analyse (OOA)	Analysemodell mit Fachklassenmodell und Zustandsmodell Auftrag
4	Aufgabe 4: Objektorientiertes Design (OOD)	Designmodell mit 3-Schichtenarchitektur, Repository, Factory und Sequenzdiagramm
5	Aufgabe 5: Objektorientierte Implementation / Programmierung (OOP)	Java-Prototyp für die fachliche Klasse Auftrag
6	Transparenzhinweis KI	Ausweis über die Benutzung von KI

Dokumentstatus

Angabe	Wert
Schule / Lehrgang	Juventus Schule HF Zürich / HF Informatik Applikationsentwicklung
Fach	Software Engineering 1
Dozent	Arif Chughtai
Haupttitel	WerkstattPilot
Arbeitsgruppe	Arbeitsgruppe 3 / WerkstattPilot-Team
Autoren	Livio Luna, Dominic Lolos
Klasse	TS TSI 2502 A
Status	Finale Abgabeverision

Version	Datum	Änderung	Verantwortlich
1.0	05.06.2026	Einzeldokumente zu Aufgabe 1 bis 5 erstellt und geprüft.	Arbeitsgruppe 3
1.1	06.06.2026	Textreview, Prüfungsabdeckung und Abgabecheck durchgeführt.	Arbeitsgruppe 3
2.0	10.06.2026	Finales Gesamtdokument erstellt, Code geprüft, Arbeitsprotokoll bereinigt und Javadoc erzeugt.	Arbeitsgruppe 3

1 Aufgabe 1: Projektauftrag

Projektauftrag und priorisierte Anforderungen für WerkstattPilot

Projektauftrag und fachlicher Umfang

1.1 Prüfungsabdeckung Aufgabe 1

Prüfziel / Bewertungsgegenstand	Abdeckung	Status	Umsetzung in der finalen Abgabe
Projektauftrag / Ausgangslage	Kapitel 1.2	erfüllt	Ausgangslage aus Aufgabe 1 übernommen.
Ziele / Nutzen	Kapitel 1.3	erfüllt	Ziele, Mission und Vision mit Iteration 1 verbunden.
Stakeholder	Kapitel 1.4	erfüllt	Stakeholder aus Aufgabe 1 übernommen und konsistent benannt.
Use Cases priorisieren	Kapitel 1.5 und 1.6	erfüllt	UC1 und UC2 als Umfang der ersten Iteration begründet.
Nicht-funktionale Anforderungen / Kontext	Kapitel 1.7	erfüllt	Zielbild lokaler Datenhaltung und Prototyp mit Mock-DB klar abgegrenzt.

1.2 Problemstellung und Ausgangslage

WerkstattPilot adressiert typische Medienbrüche in KFZ-Werkstätten. Aufträge, Arbeitszeiten und Teile werden häufig mit Papier und Excel erfasst. Dadurch entstehen doppelte Erfassungen, manuelle Datenübertragungen, Zeitverlust und Fehler.

Zusätzlich fehlt ohne ein einheitliches System der laufende Überblick über Auftragsstatus, Termine und Lagerbestand. Die Anwendung soll diesen Ablauf lokal, einfach und nachvollziehbar digitalisieren.

Problem	Beschreibung
Medienbrüche	Aufträge, Arbeitszeiten und Teile werden mit Papier und Excel erfasst.
Doppelte Erfassung	Manuelle Datenübertragung führt zu Zeitverlust und Fehlern.
Fehlender Überblick	Es fehlt Transparenz über Auftragsstatus, Termine und Lagerbestand.

1.3 Ziele, Mission und Vision

Ziel	Beschreibung
Schnelle Auftragserfassung	Einheitliche Erfassung von Kunde, Fahrzeug und Beanstandung.
Transparenz	Überblick über alle Auftragsstatus in Echtzeit.
Lagerführung	Nachvollziehbarer Teileverbrauch und Bestandsführung.
Dokumentation	Belege lokal als PDF oder zum Drucken.

Element	Beschreibung
Mission	Werkstattaufträge und Teileflüsse lokal, einfach und fehlerarm digitalisieren.
Vision	Jede/r Mitarbeitende sieht jederzeit den aktuellen Stand und die nächsten Schritte je Auftrag.

1.4 Stakeholder

Gruppe	Stakeholder	Relevanz
Kunden / Auftraggeber	Werkstattinhaber/in, Serviceleitung	Definieren fachliche Erwartungen, Nutzen und Prioritäten.
Benutzer	Serviceberater/in, Mechaniker/in	Arbeiten mit Auftragserfassung, Auftragsübersicht und Statuswechsel.
Unterstützende Stakeholder	Buchhaltung, Teilelieferant/Logistik	Sind für spätere Funktionen wie Rechnung, Mahnwesen, Bestellung und Lieferung relevant.

1.5 Use Cases priorisiert

Priorität	Use Case	Beschreibung	Iteration
1	Auftrag anlegen	Kunde, Fahrzeug, Beanstandung und Termin erfassen.	Iteration 1
2	Auftragsübersicht & Status	Übersicht anzeigen und Status ändern, zum Beispiel in Arbeit, wartet auf Teile oder fertig.	Iteration 1
3	Arbeitspositionen erfassen	Leistung, Dauer, Preis oder Ansatz dokumentieren.	später
4	Teile buchen	Teile aus Lager zu Auftrag buchen.	später
5	Lagerbestand verwalten	Teil anlegen, Bestand erhöhen oder vermindern, Mindestbestand definieren.	später
6	PDF-Belege erzeugen	Rechnung und Arbeitsauftrag als PDF erzeugen und drucken.	später
7	Suche / Filter	Auftrag nach Kunde, Kennzeichen, Status oder Datum suchen.	später

1.6 Umfang der ersten Iteration

Die erste Iteration umfasst UC1 Auftrag anlegen und UC2 Auftragsübersicht & Status. Die Reihenfolge der Use Cases basiert auf Business Value und Implementierungsaufwand. UC1 und UC2 liefern einen hohen Nutzen für Auftragsabwicklung, Transparenz und Effizienz bei gleichzeitig überschaubarem technischen Aufwand.

Use Case	Inhalt der ersten Iteration
UC1 Auftrag anlegen	Kunde, Fahrzeug, Beanstandung und Termin in einem Formular bzw. Prototyp-Ablauf erfassen.
UC2 Auftragsübersicht & Status	Zentrale Übersicht aller Aufträge mit Statuswechsel in wenigen Schritten.

Abgrenzung: Aufgabe 1 beschreibt das fachliche Zielbild. Die Implementation in Aufgabe 5 ist ein bewusst kleiner Java-Prototyp mit ConsoleClient und Mock-DB. Eine GUI und eine echte lokale Datenbank gehören nicht zum aktuellen Codeumfang.

1.7 Nicht-funktionale Anforderungen und Systemkontext

Kategorie	Anforderung
Rahmenbedingungen	Keine Cloud und keine externen Webservices.
Datenhaltung	Zielbild: lokale Datenhaltung, zum Beispiel SQLite oder PostgreSQL. Prototyp: Mock-DB im Hauptspeicher.
Betrieb	Windows-PC im Werkstattnetz, offline-fähig, LAN optional.
Usability	Auftragserfassung maximal 2 Minuten, Statuswechsel maximal 3 Klicks.
Zuverlässigkeit	Keine verlorenen Änderungen und saubere Validierung.

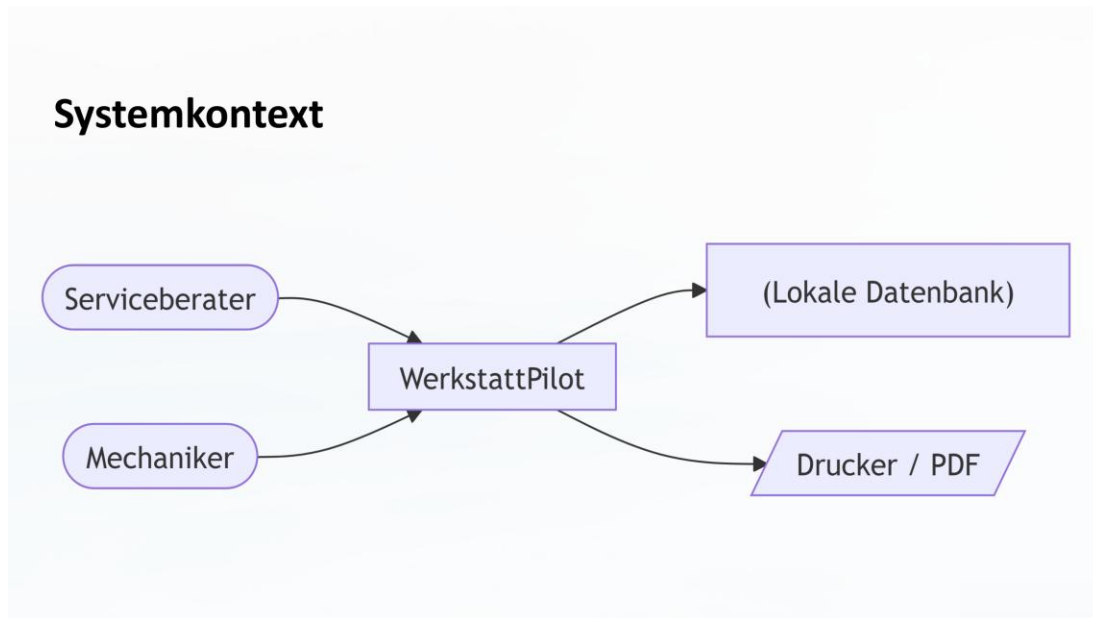


Abbildung 1: Systemkontext aus Aufgabe 1

1.8 Projektplan Iteration 1

Phase	Zeitraum	Inhalt
1	Woche 1-2	Systemarchitektur und Datenmodell.
2	Woche 3-4	UC1 Auftrag anlegen.
3	Woche 5-6	UC2 Auftragsübersicht und Status.
4	Woche 7	Integration, Testing und Dokumentation.

Lieferumfang der ersten Iteration ist eine lauffähige Anwendung mit Auftragserfassung, Auftragsübersicht und Statusmanagement für einen kleinen Werkstattkontext.

2 Aufgabe 2: Projektumgebung definieren und dokumentieren

Gemeinsame Projektumgebung, Ordnerstruktur und Entwicklungswerkzeuge

Projektumgebung, Arbeitsweise und Werkzeuge

2.1 Prüfungsabdeckung Aufgabe 2

Prüfziel / Bewertungsgegenstand	Abdeckung	Status	Umsetzung in der finalen Abgabe
Richtlinien Schreibweisen / Schreibstil	Kapitel 2.3	erfüllt	Einheitliche Begriffe, Dateinamen, Java- und UML-Namensregeln dokumentiert.
Ordnerstruktur	Kapitel 2.4	erfüllt	Projektordner und Zweck der Hauptordner beschrieben.
Organisation Arbeitsgruppe	Kapitel 2.5	erfüllt	Beide Personen bearbeiten alle Aufgaben; Schwerpunkte sind ergänzend formuliert.
Entwicklungssoftware und Installation	Kapitel 2.7 und 2.8	erfüllt	Eclipse, JDK, Git, Visual Paradigm, Word und Excel einheitlich dokumentiert.
Entscheidungen und Glossar	Kapitel 2.9 und 2.10	erfüllt	Projektentscheide und Begriffe aktualisiert.
Dozentenfeedback / Arbeitsprotokoll	Kapitel 2.11 und 2.12	erfüllt	Finales Arbeitsprotokoll wird als separates Abgabedokument mitgegeben.

2.2 Dokumentzweck und Gültigkeit

Dieses Kapitel beschreibt die Projektumgebung für WerkstattPilot. Es definiert die gemeinsame Arbeitsweise, die eingesetzten Werkzeuge, die Ordnerstruktur, die wichtigsten Projektentscheidungen und die zentralen Fachbegriffe.

Die Projektumgebung gilt für alle Arbeitsergebnisse im Projekt. Dazu gehören Projektdokumentation, UML-Modelle, Java-Quelltexte, Testdokumentation, Issue Tracking, Installationshinweise und Abgabeunterlagen.

Für die erste Iteration werden die beiden wichtigsten fachlichen Anforderungen umgesetzt: Auftrag anlegen sowie Auftragsübersicht mit Statusänderung. Lagerverwaltung, Arbeitspositionen und PDF-Belege sind spätere Iterationen.

2.3 Richtlinien für Schreibweisen und Schreibstil

- Die Projektdokumentation wird in Deutsch verfasst.
- Der Schreibstil ist kurz, formal und nachvollziehbar.
- Fachbegriffe werden einheitlich verwendet.
- Entscheidungen, Annahmen und offene Punkte werden sichtbar dokumentiert.
- Rechtschreibung, Gross- und Kleinschreibung sowie Java- und UML-Bezeichnungen werden vor der Abgabe geprüft.

Element	Regel	Beispiel
Ordner	nummeriert, klein, mit Unterstrich	02_dokumentation, 03_entwicklung
Dokumente	klein, Projektname, Aufgabe, Inhalt, Version	werkstattpilot_aufgabe2_projektumgebung_v1.0.docx
Diagramme	Diagrammtyp und Inhalt im Dateinamen	klassendiagramm_auftrag_v1.0.png
Versionen	Version mit vX.Y kennzeichnen	v1.0, v1.1
Abgaben	finale Dateien im Abgabeordner ablegen	arbeitsgruppe3-projektarbeit.pdf

Element	Regel	Beispiel
Packages	lowercase, keine Leerzeichen	ch.werkstattteam.werkstattpilot.business
Klassen	PascalCase, Singular	Auftrag, Kunde, Fahrzeug
Interfaces	PascalCase, fachlich klar	AuftragRepository
Methoden	camelCase, Verb plus Objekt	auftragAnlegen(), statusAendern()
Attribute	camelCase, fachlich eindeutig	auftragsNummer, kundenName
Konstanten	GROSSBUCHSTABEN_MIT_UNTERSTRICH	MAX_AKTIVE_AUFTRAEGE
UML-Klassen	Substantive in Kundensprache	Auftrag, Fahrzeug
Use Cases	Verb plus Objekt	Auftrag anlegen

Die fachlichen Klassen und Begriffe werden auf Deutsch benannt, weil Projekt und Aufgabenstellung in Deutsch geführt werden. Technische Standardbegriffe wie Repository, Factory, Mock-DB oder Unit-Test werden beibehalten, wenn sie im Unterricht oder in Java üblich sind.

2.4 Ordnerstruktur

Die Ordnerstruktur trennt Installation, Dokumentation, Entwicklung, Issue Tracking, Versionen, Abgaben und Backups.

```

WerkstattPilot/
01_installation/
  java_jdk/
  eclipse/
  visual_paradigm/
  git/
02_dokumentation/
  aufgabe_1_projektauftrag/
  aufgabe_2_projektumgebung/
  aufgabe_3_analyse/
  aufgabe_4_design/
  aufgabe_5_implementation/
  diagramme/
03_entwicklung/
  werkstattPilot/
    src/main/java/ch/werkstattteam/werkstattpilot/
      presentation/
      business/
      persistence/
    src/test/java/ch/werkstattteam/werkstattpilot/test/
      business/
      persistence/
04_issue_tracking/
05_versionen/
06_abgabe/
07_backup/

```

Ordner	Zweck
01_installation	Installationsdateien, Setup-Hinweise und Konfigurationsnotizen.
02_dokumentation	Projektauftrag, Projektumgebung, Analyse, Design, Implementation, Diagramme und Protokolle.
03_entwicklung	Eclipse-Projekt mit Java-Quelltexten, Packages und Tests.
04_issue_tracking	Excel-Datei für Aufgaben, offene Punkte, Fehler und Status.
05_versionen	Zwischenstände und wichtige freigegebene Versionen.
06_abgabe	Finale DOCX-, PDF-, Präsentations-, Quelltext- und Arbeitsprotokoll-Dateien.
07_backup	Sicherungen wichtiger Projektstände.

2.4.1 Versionsverwaltung und Git-Repository

Die Versionsverwaltung des Projekts wurde mit Git und GitHub geführt. Das Repository dient als zentrale Ablage für den Java-Prototyp, die Projektstruktur und die technische Nachvollziehbarkeit der Implementierung. Für die finale Abgabe wurde der geprüfte Stand zusätzlich als Eclipse-Projekt-ZIP abgelegt, da gemäss Abgaberichtlinien das Eclipse-Projekt als ZIP-Datei abzugeben ist. Das GitHub-Repository bleibt als ergänzende Referenz zur Entwicklungshistorie bestehen.

Angabe	Wert	Angabe
Repository	https://github.com/6ix9ine4our20wenty/HF_Juventus_SoftwareEntwicklung_WerkstattPilot	Repository
Branch	main	Branch
Projektordner	WerkstattPilot	Projektordner
Hauptklasse	ch.werkstattteam.werkstattpilot.presentation.WerkstattPilotApp	Hauptklasse
Testklassen	ch.werkstattteam.werkstattpilot.test.persistence.AuftragRepositoryTest ch.werkstattteam.werkstattpilot.test.business.AuftragServiceTest	Testklassen
Hinweis	Das Repository ergänzt die Abgabe. Bewertungsrelevant bleibt das abgegebene Eclipse-Projekt-ZIP.	Hinweis

2.5 Organisation der Arbeitsgruppe

Die Arbeitsgruppe arbeitet als Zweierteam. Beide Mitglieder bearbeiten alle Aufgaben mit, prüfen die Ergebnisse gegenseitig und stimmen wichtige Entscheidungen gemeinsam ab. Die Schwerpunkte dienen nur der Arbeitsteilung und ersetzen keine gemeinsame Verantwortung.

Person	Schwerpunkt	Verantwortung
Livio Luna	Dokumentation, Struktur, Abgabecheck und Java-Unterstützung	Erstellt und prüft Projektdokumente, kontrolliert Abgabekriterien und unterstützt die technische Abstimmung.
Dominic Lolos	Code, technische Umsetzung, UML und Gegenreview	Prüft Java-Prototyp, Diagramme, technische Struktur und unterstützt die Dokumentation.
Beide	Abstimmung, Review, Präsentation, finale Abgabe	Stimmen Inhalte ab, prüfen Konsistenz und präsentieren die Projektarbeit gemeinsam.

Bereich	Vorgehen
Kommunikation	Teams, persönliche Abstimmung, kurze Reviews.
Austausch von Arbeitsergebnissen	Git für Code, gemeinsamer Projektordner für Dokumente und Abgaben.
Planung	Arbeitsprotokoll und Issue Tracking in Excel.
Qualitätssicherung	Gegenseitiger Review, Code-Run, Testlauf und Prüfung der Bewertungskriterien.
Dokumentation von Entscheiden	Entscheidungen werden mit Begründung und Auswirkung dokumentiert.

2.6 Grober Projektplan

Phase	Inhalt	Status
Aufgabe 1	Projektauftrag und Anforderungen	abgeschlossen
Aufgabe 2	Projektumgebung definieren und dokumentieren	abgeschlossen
Aufgabe 3	Objektorientierte Analyse: Fachklassenmodell und Zustandsmodell	abgeschlossen
Aufgabe 4	Objektorientiertes Design: 3-Schichtenmodell, Patterns, Sequenzdiagramm	abgeschlossen
Aufgabe 5	Java-Prototyp, Mock-DB, Unit-Tests, Javadoc und Endbenutzertest	abgeschlossen

2.7 Eingesetzte Entwicklungsssoftware

Software	Version / Stand	Zweck	Installationsort	Website / Doku
Java JDK	21 LTS	Programmiersprache und Laufzeitumgebung für den Prototyp	lokaler Rechner	oracle.com/java oder adoptium.net
Eclipse IDE	lokal installierte Version	Entwicklungsumgebung für Java-Projekt, Packages und Tests	lokaler Rechner	eclipse.org

Git	2.x / lokale Version	Versionierung von Quelltext und wichtigen Projektständen	lokaler Rechner	git-scm.com
Visual Paradigm	lokal installierte Version	Erstellung von UML-Diagrammen	lokaler Rechner	visual-paradigm.com
Microsoft Word	Microsoft 365 / lokal	Projektdokumentation und Abgabe als DOCX/PDF	lokaler Rechner	support.microsoft.com
Microsoft Excel	Microsoft 365 / lokal	Issue Tracking, Arbeitsprotokoll und offene Punkte	lokaler Rechner	support.microsoft.com

Für die Tests wird bewusst kein JUnit verwendet. Die Testklassen sind einfache Java-Klassen mit main-Methode und eigener pruefe-Methode.

2.8 Installation und Konfiguration

Bereich	Schritt	Status
Java JDK	JDK 21 installieren, Projekt mit JRE_CONTAINER verbinden, java -version prüfen.	erledigt
Eclipse	Eclipse starten, Projekt importieren, .project und .classpath verwenden.	erledigt
Eclipse	Source-Folder src/main/java und src/test/java prüfen.	erledigt
Git	Repository bzw. Projektordner prüfen, sinnvolle Zwischenstände sichern.	erledigt
Visual Paradigm	UML-Projekt WerkstattPilot pflegen und Diagramme als PNG exportieren.	erledigt
Word / Excel	Einheitliche Dokumente, Tabellen, Arbeitsprotokoll und finale PDF-Dateien erstellen.	erledigt

2.9 Projektscheidungen

Datum	Entscheid	Begründung	Auswirkung
05.06.2026	Projektname WerkstattPilot	Der Name beschreibt den fachlichen Zweck der Anwendung klar.	Einheitlicher Name in Dokumenten, Diagrammen und Code.
05.06.2026	Iteration 1 umfasst UC1 und UC2	Auftrag anlegen und Statusübersicht haben hohen fachlichen Nutzen.	Analyse, Design und Implementation fokussieren auf diese Funktionen.
05.06.2026	Deutsch als Fachsprache	Aufgabenstellung und Fachdomäne sind in Deutsch.	Klassen und Fachbegriffe werden auf Deutsch benannt.
05.06.2026	Java und Eclipse verwenden	Java passt zum Unterricht; Eclipse-Projekt entspricht den Abgabevorgaben.	Prototyp wird als einfaches Java-Projekt umgesetzt.
05.06.2026	Keine echte Datenbank in Iteration 1	Der Prototyp soll einfach und testbar bleiben.	Datenhaltung erfolgt über Mock-DB bzw. Java Collections im Hauptspeicher.
05.06.2026	Einfache Java-Tests ohne externe Bibliothek	Basic Java bleibt nachvollziehbar und passend zum Umfang.	Testklassen besitzen main-Methode und pruefe-Methode.
05.06.2026	Visual Paradigm für UML	Das Tool unterstützt die geforderten UML-Diagramme.	Diagramme werden dort bzw. codekonform als PNG erstellt.
10.06.2026	Finaler Code-Abgleich	Dokumentation, Diagramme und Code müssen übereinstimmen.	Methoden und Diagramme wurden auf den aktuellen Java-Code angepasst.

2.10 Glossar

Begriff	Beschreibung	Synonyme
WerkstattPilot	Name des zu entwickelnden Softwaresystems für digitale Werkstattaufträge.	Anwendung, System
Auftrag	Fachliches Objekt, das einen Werkstattauftrag für ein Fahrzeug beschreibt.	Werkstattauftrag
Kunde	Person oder Organisation, die einen Auftrag für ein Fahrzeug erteilt.	Auftraggeber
Fahrzeug	Fahrzeug des Kunden, für das ein Auftrag erfasst wird.	Auto, Kundenfahrzeug
Beanstandung	Vom Kunden gemeldetes Problem oder gewünschte Arbeit am Fahrzeug.	Problem, Anliegen
Termin	Geplanter Zeitpunkt für Annahme oder Bearbeitung des Auftrags.	Datum, Werkstatttermin
Status	Bearbeitungszustand eines Auftrags, zum Beispiel offen, in Arbeit oder fertig.	Auftragsstatus
Auftragsübersicht	Ansicht aller relevanten Aufträge mit Status und wichtigen Informationen.	Übersicht
Iteration 1	Erste abgegrenzte Umsetzungsphase mit Auftrag anlegen und Auftragsübersicht.	erste Lieferung
Use Case	Beschreibt eine fachliche Funktion aus Sicht eines Benutzers.	Anwendungsfall
Mock-DB	Vereinfachte Datenhaltung ohne echte Datenbank, zum Beispiel im Arbeitsspeicher.	In-Memory-Datenhaltung
Presentation-Schicht	Schicht für Benutzerschnittstelle oder Konsolenausgabe.	UI-Schicht
Business-Schicht	Schicht für Fachlogik und fachliche Regeln.	Logikschicht
Persistence-Schicht	Schicht für Speichern und Lesen von fachlichen Objekten.	Datenzugriffsschicht
Repository	Schnittstelle oder Klasse für den Zugriff auf gespeicherte fachliche Objekte.	Datenzugriff
Factory	Erzeugt Objekte oder konkrete Implementierungen zentral.	Erzeugerklasse
Javadoc	Dokumentationsstandard für Java-Quelltext und API-Dokumentation.	Java-Dokumentation

2.11 Rückmeldungen des Dozenten

Datum	Rückmeldung	Umsetzung	Status
17.03.2026	Keine Rückmeldung zu Aufgabe 2	Keine Anpassung notwendig.	abgeschlossen

2.12 Arbeitsprotokoll und Quellen

Das Arbeitsprotokoll wird separat als arbeitsgruppe3-arbeitsprotokoll.xlsx abgegeben. Es enthält die final bereinigten Tätigkeiten pro Aufgabe und weist beide Personen über die gesamte Projektarbeit aus.

Quelle	Verwendung im Projekt
Aufgabenblatt Aufgabe 1	Projektauftrag und Anforderungen für WerkstattPilot.
Aufgabenblatt Aufgabe 2	Vorgaben zur Projektumgebung, Bewertungskriterien und Dokumentationsumfang.
Aufgabenblatt Aufgabe 5	Vorgaben zur Java-Implementation, 3-Schichtenstruktur und Mock-DB.
Java-Dokumentation	Referenz für Java-Sprache, JDK und Javadoc.
Eclipse-Dokumentation	Referenz für Projektimport und Java-Entwicklung.
Git-Dokumentation	Referenz für Versionierung und grundlegende Git-Befehle.
Visual Paradigm Dokumentation	Referenz für Erstellung und Export von UML-Diagrammen.

3 Aufgabe 3: Objektorientierte Analyse (OOA)

Analysemodell mit Fachklassenmodell und Zustandsmodell Auftrag

Analysemodell in Kundensprache

3.1 Prüfungsabdeckung Aufgabe 3

Prüfziel / Bewertungsgegenstand	Abdeckung	Status	Umsetzung in der finalen Abgabe
Fachklassenmodell	Kapitel 3.5	erfüllt	Diagramm gut lesbar eingebunden.
Attribute und Assoziationen	Kapitel 3.5.1	erfüllt	Multiplizitäten und fachliche Bedeutung beschrieben.
Vererbung prüfen	Kapitel 3.4.1	erfüllt	Vererbung bewusst nicht verwendet, da keine Spezialisierung nötig ist.
Zustandsmodell	Kapitel 3.6	erfüllt	Statuswerte mit Code-Enum abgeglichen.
Glossar Verantwortlichkeiten	Kapitel 3.7	erfüllt	Fachliche Verantwortlichkeiten ohne UML-Theorie beschrieben.
Dozentenfeedback / Arbeitsprotokoll	Kapitel 3.8 und 3.9	erfüllt	Feedback und Verweis auf finales Arbeitsprotokoll enthalten.

3.2 Ziel und Grundlage der Analyse

Dieses Kapitel beschreibt das objektorientierte Analysemodell für WerkstattPilot. Das Modell basiert auf dem Projektauftrag und auf den beiden priorisierten funktionalen Anforderungen der ersten Iteration: Auftrag anlegen sowie Auftragsübersicht mit Statusänderung.

Die Analyse verwendet die Kundensprache Deutsch. Die Klassen beschreiben fachliche Gegenstände der WerkstattDomäne und enthalten deshalb nur fachliche Attribute und Beziehungen. Methoden werden im Fachklassenmodell bewusst nicht aufgeführt.

Grundlage	Bedeutung für Aufgabe 3
Aufgabe 1 - Projektauftrag	Beschreibt Problemstellung, Ziele, Stakeholder, priorisierte Use Cases und Nicht-funktionale Anforderungen.
Aufgabe 2 - Projektumgebung	Legt fest, dass Iteration 1 auf UC1 Auftrag anlegen und UC2 Auftragsübersicht mit Statusänderung fokussiert.
Aufgabe 3 - Aufgabenstellung	Verlangt Fachklassenmodell mit Attributen und Assoziationen, Prüfung von Vererbung, Zustandsmodell und Glossar.

3.3 Umfang und Annahmen

Für die objektorientierte Analyse werden nur die fachlichen Inhalte modelliert, die für die erste Iteration benötigt werden. Der Umfang bleibt bewusst klein, damit Design und Prototyp nachvollziehbar umgesetzt werden können.

Use Case	Fachlicher Inhalt	Auswirkung auf das Analysemodell
UC1 Auftrag anlegen	Kunde, Fahrzeug, Beanstandung und Termin erfassen.	Die Klassen Kunde, Fahrzeug, Auftrag und Beanstandung werden benötigt.
UC2 Auftragsübersicht & Status	Aufträge anzeigen und Status ändern.	Die Klassen Auftrag und Auftragsstatus werden benötigt; für Auftrag wird ein Zustandsmodell erstellt.

Annahme	Begründung
Ein Auftrag gehört genau zu einem Kunden.	Der Auftrag wird bei der Annahme für einen konkreten Auftraggeber erfasst.
Ein Auftrag gehört genau zu einem Fahrzeug.	Beanstandung und Werkstatttermin beziehen sich auf ein konkretes Fahrzeug.
Ein Kunde kann mehrere Fahrzeuge besitzen.	Ein Auftraggeber kann über die Zeit mehrere Fahrzeuge in die Werkstatt bringen.
Ein Fahrzeug kann mehrere Aufträge haben.	Für dasselbe Fahrzeug können wiederholt Werkstattaufträge entstehen.
Ein Auftrag enthält mindestens eine Beanstandung.	Beim Anlegen eines Auftrags muss mindestens ein Problem oder Anliegen erfasst werden.
Ein Auftrag hat zu jedem Zeitpunkt genau einen aktuellen Status.	Die Auftragsübersicht braucht einen eindeutigen Bearbeitungsstand.

3.4 Klassenkandidaten und Abgrenzung

Aus der Problemstellung und den priorisierten Use Cases ergeben sich mehrere mögliche Klassenkandidaten. In das Fachklassenmodell werden nur Klassen aufgenommen, die direkt für UC1 und UC2 gebraucht werden.

Klassenkandidat	Entscheid	Begründung
Kunde	aufnehmen	Wird für die Auftragserfassung benötigt und ist Auftraggeber des Werkstattauftrags.
Fahrzeug	aufnehmen	Wird zusammen mit dem Auftrag erfasst und ist fachlich zentral für die Werkstattarbeit.
Auftrag	aufnehmen	Zentrale Klasse der ersten Iteration und Grundlage für Statusübersicht.
Beanstandung	aufnehmen	Beschreibt das gemeldete Problem oder Anliegen des Kunden.
Auftragsstatus	aufnehmen	Wird für Übersicht und Statuswechsel benötigt.
Arbeitsposition	zurückstellen	Erst für spätere Leistungs- und Preispositionen relevant.
Lagerartikel	zurückstellen	Erst für Teilebuchung und Lagerbestand relevant.
Rechnung	zurückstellen	Erst für PDF-Belege und Abrechnung relevant.
Mitarbeiter	zurückstellen	In Iteration 1 wird kein Benutzerkonzept modelliert.

3.4.1 Prüfung Vererbung

Der Einsatz von Vererbung wurde geprüft. Für die erste Iteration ist keine Vererbung sinnvoll, weil keine fachlichen Spezialisierungen benötigt werden. Kunde, Fahrzeug, Auftrag, Beanstandung und Auftragsstatus haben unterschiedliche Verantwortlichkeiten und stehen über Assoziationen miteinander in Beziehung.

Mögliche Vererbung	Entscheid
Person als Oberklasse für Kunde	Nicht verwenden. In Iteration 1 gibt es keine weiteren Personentypen wie Mitarbeiter oder Lieferant.
Fahrzeug-Unterklassen nach Typ	Nicht verwenden. Die erste Iteration braucht keine Unterscheidung nach Fahrzeugarten.
Auftrag-Unterklassen nach Status	Nicht verwenden. Der Status wird fachlich über Auftragsstatus bzw. das Zustandsmodell abgebildet.

3.5 Fachklassenmodell

Das Fachklassenmodell zeigt die Klassen der ersten Iteration mit ihren wichtigsten Attributen und Assoziationen. Es enthält keine Methoden, da diese erst im Designmodell festgelegt werden.

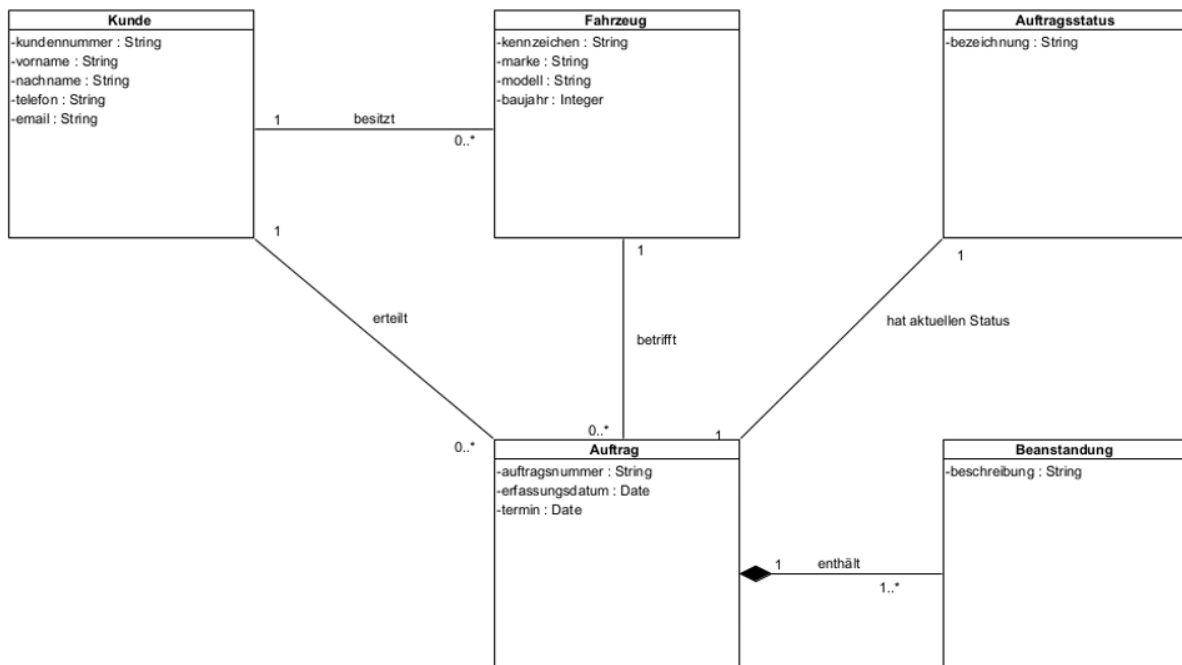


Abbildung 2: Fachklassenmodell WerkstattPilot, Iteration 1

3.5.1 Assoziationen und Multiplizitäten

Beziehung	Multiplizität	Fachliche Bedeutung
Kunde - Fahrzeug	Kunde 1 zu Fahrzeug 0..*	Ein Kunde kann kein, ein oder mehrere Fahrzeuge besitzen. Ein Fahrzeug gehört im Modell zu genau einem Kunden.
Kunde - Auftrag	Kunde 1 zu Auftrag 0..*	Ein Kunde kann mehrere Aufträge erteilen. Jeder Auftrag gehört zu genau einem Kunden.
Fahrzeug - Auftrag	Fahrzeug 1 zu Auftrag 0..*	Ein Fahrzeug kann über die Zeit mehrere Aufträge haben. Jeder Auftrag betrifft genau ein Fahrzeug.
Auftrag - Beanstandung	Auftrag 1 zu Beanstandung 1..*	Ein Auftrag enthält mindestens eine Beanstandung. Mehrere Beanstandungen pro Auftrag sind möglich.
Auftrag - Auftragsstatus	Auftrag 1 zu Auftragsstatus 1	Ein Auftrag hat genau einen aktuellen Status.

3.5.2 Verantwortlichkeitsprüfung

Prüfpunkt	Ergebnis
Kann UC1 Auftrag anlegen abgebildet werden?	Ja. Kunde, Fahrzeug, Beanstandung, Termin und Auftrag sind im Modell vorhanden.
Kann UC2 Auftragsübersicht & Status abgebildet werden?	Ja. Auftrag und Auftragsstatus erlauben die fachliche Anzeige und Statusänderung.
Ist das Modell auf Iteration 1 begrenzt?	Ja. Spätere Themen wie Lager, Rechnung und Arbeitspositionen sind bewusst ausgeschlossen.
Sind Methoden im Analysemodell vermieden?	Ja. Das Fachklassenmodell enthält nur Klassen, Attribute und Assoziationen.

3.6 Zustandsmodell Auftrag

Für das Zustandsmodell wird die Klasse Auftrag verwendet, weil sie die zentrale Klasse der ersten Iteration ist. Der Zustand eines Auftrags ist für die Auftragsübersicht und für den Statuswechsel direkt relevant.

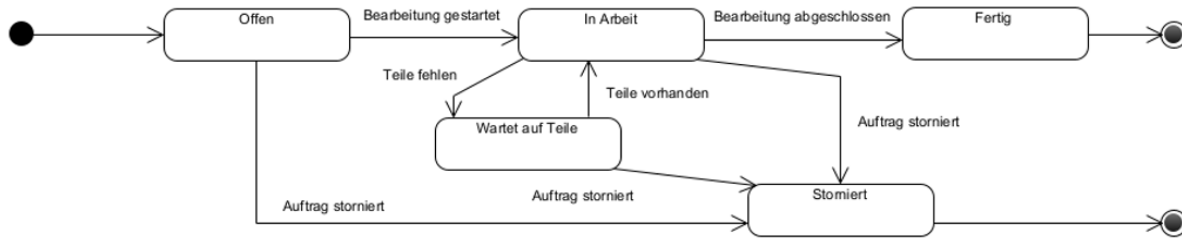


Abbildung 3: Zustandsmodell der Klasse Auftrag

Zustand	Beschreibung
Offen	Der Auftrag wurde erfasst, aber die Bearbeitung wurde noch nicht gestartet.
In Arbeit	Die Werkstatt bearbeitet den Auftrag aktiv.
Wartet auf Teile	Die Bearbeitung ist pausiert, weil benötigte Teile fehlen.
Fertig	Die fachliche Bearbeitung des Auftrags ist abgeschlossen.
Storniert	Der Auftrag wurde abgebrochen und wird nicht weiter bearbeitet.

Prüfung	Ergebnis
Benötigt Auftrag einen eindeutigen Status?	Ja. Deshalb wird Auftragsstatus im Fachklassenmodell geführt.
Sind alle Zustände aus UC2 abgedeckt?	Ja. Offen, In Arbeit, Wartet auf Teile und Fertig decken den fachlichen Ablauf ab; Storniert ergänzt den Abbruchfall.
Muss das Fachklassenmodell ergänzt werden?	Nein. Die benötigten Klassen und Beziehungen sind bereits vorhanden.

3.7 Glossar der fachlichen Verantwortlichkeiten

Das Glossar beschreibt die fachliche Verantwortung jeder Klasse im Analysemodell. Es wiederholt nicht einfach die Attribute aus dem Klassendiagramm, sondern erklärt die Rolle der Klasse im Fachmodell.

Klasse	Fachliche Verantwortlichkeit	Wichtige Abgrenzung
Kunde	Repräsentiert den Auftraggeber eines Werkstattauftrags und bündelt Kundendaten für Auftragserfassung und Kontaktaufnahme.	Nicht zuständig für Benutzerrechte, Login oder interne Mitarbeitende.
Fahrzeug	Repräsentiert das Fahrzeug, für das ein Auftrag eröffnet wird, und stellt den fachlichen Bezug zwischen Kunde und Auftrag her.	Nicht zuständig für Lagerteile, Arbeitspositionen oder Abrechnung.
Auftrag	Zentrale Klasse der ersten Iteration. Sie verbindet Kunde, Fahrzeug, Beanstandung, Termin und aktuellen Status zu einem Werkstattauftrag.	Nicht zuständig für technische Speicherung oder GUI-Darstellung.
Beanstandung	Beschreibt das vom Kunden gemeldete Problem oder Anliegen.	Nicht zuständig für spätere Arbeitspositionen oder Rechnungspositionen.
Auftragsstatus	Beschreibt den aktuellen Bearbeitungsstand eines Auftrags.	Nicht als eigene Auftrag-Unterklasse modelliert.

Begriff	Beschreibung	Synonyme
Auftrag	Werkstattauftrag, der für ein Fahrzeug erfasst und über einen Status verfolgt wird.	Werkstattauftrag
Kunde	Auftraggeber des Werkstattauftrags.	Auftraggeber
Fahrzeug	Kundenfahrzeug, das im Auftrag bearbeitet wird.	Auto, Kundenfahrzeug
Beanstandung	Vom Kunden gemeldetes Problem oder gewünschte Arbeit.	Problem, Anliegen
Termin	Geplanter Zeitpunkt für Annahme oder Bearbeitung.	Werkstattermin
Status	Aktueller Bearbeitungszustand eines Auftrags.	Auftragsstatus

3.8 Rückmeldungen des Dozenten

Datum	Rückmeldung	Umsetzung / Einarbeitung	Status
Di, 21.04.2026	Der Dozent hat gesagt, dass wir auf dem guten Weg sind.	Der gewählte Projektumfang wurde beibehalten. Das Analysemodell wurde weiterhin auf Iteration 1 fokussiert und nicht mit späteren Funktionen überladen.	umgesetzt

3.9 Arbeitsprotokoll und Quellen

Die detaillierten Arbeitseinträge zu Aufgabe 3 sind im finalen Arbeitsprotokoll enthalten.

Quelle	Verwendung im Dokument
Aufgabe 1 - WerkstattPilot Projektauftrag	Grundlage für Problemstellung, Ziele, Stakeholder und priorisierte Use Cases.
Aufgabe 2 - Projektumgebung	Grundlage für Umfang der ersten Iteration und Projektentscheide.
Aufgabe 3 - Objektorientierte Analyse	Vorgaben für Fachklassenmodell, Zustandsmodell, Glossar und Bewertungskriterien.
OpenOLAT Unterrichtsstoff 1- Analyse	Theoretische Grundlage aus dem Unterricht; keine Wiederholung der Theorie im Dokument.

4 Aufgabe 4: Objektorientiertes Design (OOD)

Designmodell mit 3-Schichtenarchitektur, Repository, Factory und Sequenzdiagramm

Designmodell und technische Struktur

4.1 Prüfungsabdeckung Aufgabe 4

Prüfziel / Bewertungsgegenstand	Abdeckung	Status	Umsetzung in der finalen Abgabe
Business-Schicht	Kapitel 4.5	erfüllt	Methoden auf aktuellen Java-Code angepasst.
Persistence-Schicht	Kapitel 4.6	erfüllt	Repository, Mock-DB und Factory dokumentiert.
Integriertes Klassendiagramm	Kapitel 4.7.2	erfüllt	Diagramm auf aktuelle Methoden und Packages angepasst.
Komponentendiagramm	Kapitel 4.7.1	erfüllt	3-Schichtenarchitektur mit technischen Abhängigkeiten dargestellt.
Sequenzdiagramm	Kapitel 4.8	erfüllt	Normalablauf UC1 mit aktuellen Methodennamen dargestellt.
Glossar / Feedback / Arbeitsprotokoll	Kapitel 4.9 bis 4.11	erfüllt	Glossar korrigiert und Arbeitsprotokoll separat finalisiert.

4.2 Ziel und Grundlage des Designmodells

Dieses Kapitel beschreibt das objektorientierte Designmodell für WerkstattPilot. Das Designmodell konkretisiert, wie die erste Iteration technisch in einer 3-Schichtenarchitektur umgesetzt wird.

Der Fokus liegt auf den beiden priorisierten Use Cases der ersten Iteration: UC1 Auftrag anlegen und UC2 Auftragsübersicht mit Statusänderung. Als zentrale fachliche Klasse wird Auftrag verwendet.

Grundlage	Bedeutung für Aufgabe 4
Aufgabe 1 - Projektauftrag	Definiert Problemstellung, Ziele, Stakeholder, priorisierte Use Cases und Rahmenbedingungen.
Aufgabe 2 - Projektumgebung	Legt Werkzeuge, Ordnerstruktur, Java-Packages, Git, Excel Issue Tracking und keine echte Datenbank für Iteration 1 fest.
Aufgabe 3 - Analysemodell	Liefert die fachlichen Klassen, Beziehungen und das Zustandsmodell des Auftrags.
Aufgabe 4 - Aufgabenstellung	Verlangt Designmodelle für Business und Persistence, integriertes 3-Schichtenmodell, Sequenzdiagramm und Glossar.
Aufgabe 5 - Implementation	Bestätigt für später: Java-Prototyp mit Mock-DB, Unit-Tests, ConsoleClient, Javadoc und Endbenutzertest.

4.3 Umfang, Annahmen und Konsistenzkorrekturen

Use Case	Design-Relevanz	Umsetzung im Designmodell
UC1 Auftrag anlegen	Auftrag mit Auftragsnummer und initialem Status erstellen.	ConsoleClient ruft AuftragService auf. AuftragServiceImpl erstellt über AuftragFactory ein Auftrag-Objekt und speichert es über AuftragRepository.
UC2 Auftragsübersicht & Status	Aufträge lesen, auf der Konsole anzeigen und den Status ändern.	AuftragService bietet Methoden zum Lesen aller Aufträge und zum Ändern des Auftragsstatus.

Annahme	Begründung
Zentrale fachliche Klasse ist Auftrag.	Auftrag verbindet die Auftragserfassung, die Übersicht und den Statuswechsel.
Die erste Implementation speichert nur die für den Prototyp nötigen Werte.	Aufgabe 5 verlangt eine kleine, testbare Implementation und keine vollständige Fachanwendung.
Es gibt keine Benutzereingaben über die Tastatur.	Der ConsoleClient arbeitet mit hart implementierten Testdaten.
Eine echte Datenbank wird nicht verwendet.	Die Persistence-Schicht arbeitet mit einer Mock-DB im Hauptspeicher.
Das Design zeigt keine Attribute.	Die Aufgabenstellung zu Aufgabe 4 verlangt Klassen und Schnittstellen, jedoch keine Attribute.
Methoden werden nur bei Schnittstellen gezeigt.	Dadurch bleibt das Design schlank und entspricht den Vorgaben der Aufgabe.

Eine frühere Projektidee mit lokaler Datenbank wurde für den aktuellen Prototyp entfernt. Das Designmodell verwendet keine MariaDB, kein JDBC und keine SQL-Abfragen. Die Datenhaltung erfolgt über AuftragRepositoryMock als In-Memory Mock-DB.

4.4 Architektur und Package-Struktur

Die Architektur folgt der vorgegebenen logischen 3-Schichtenstruktur. Die Packages orientieren sich an der dokumentierten Ordnerstruktur.

```

Basispackage: ch.werkstattteam.werkstattpilot

presentation
  WerkstattPilotApp
  ConsoleClient

business
  Auftrag
  Auftragsstatus
  AuftragFactory
  AuftragService
  AuftragServiceImpl
  AuftragServiceFactory

persistence
  AuftragRepository
  AuftragRepositoryMock
  AuftragRepositoryFactory

```

Schicht	Package	Verantwortung
Presentation	.presentation	Start der Anwendung, ConsoleClient, Anzeige von Testdaten und Resultaten auf der Konsole.
Business	.business	Fachliche Objekte, Services, Statuslogik und Factories für Aufträge.
Persistence	.persistence	Repository-Schnittstelle, Mock-Repository und gekapselte In-Memory-Datenhaltung.
Tests Business	.test.business	Einfache Java-Tests für Business-Logik in Aufgabe 5.
Tests Persistence	.test.persistence	Einfache Java-Tests für Repository und Mock-DB in Aufgabe 5.

Abhängigkeit	Regel
Presentation -> Business	ConsoleClient verwendet AuftragService bzw. AuftragServiceFactory.
Business -> Persistence	AuftragServiceImpl verwendet AuftragRepository zum Speichern und Lesen von Aufträgen.
Persistence -> Business	AuftragRepositoryMock speichert Auftrag-Objekte aus der Business-Schicht.
Keine direkte Presentation -> Persistence Abhängigkeit	Die Konsole greift nicht direkt auf RepositoryMock zu.
Keine echte DB	Persistence nutzt nur eine Collection im Hauptspeicher.

4.5 Designmodell Business-Schicht

Die Business-Schicht enthält die fachliche Koordination für die Use Cases der ersten Iteration. Der wichtigste Einstiegspunkt ist die Schnittstelle `AuftragService`. Die konkrete Klasse `AuftragServiceImpl` setzt die fachliche Logik um und verwendet `Factory` und `Repository`.

Methodenname	Rückgabe	Parameter	Zweck
auftragAnlegen	Auftrag	int auftragsNummer	Erstellt und speichert einen neuen Auftrag.
auftragLesen	Auftrag	int auftragsNummer	Liest einen Auftrag anhand der Nummer.
alleAuftraegeLesen	List<Auftrag>	-	Liest alle gespeicherten Aufträge.
statusAendern	Auftrag	int auftragsNummer, Auftragsstatus neuerStatus	Ändert den Status eines bestehenden Auftrags.

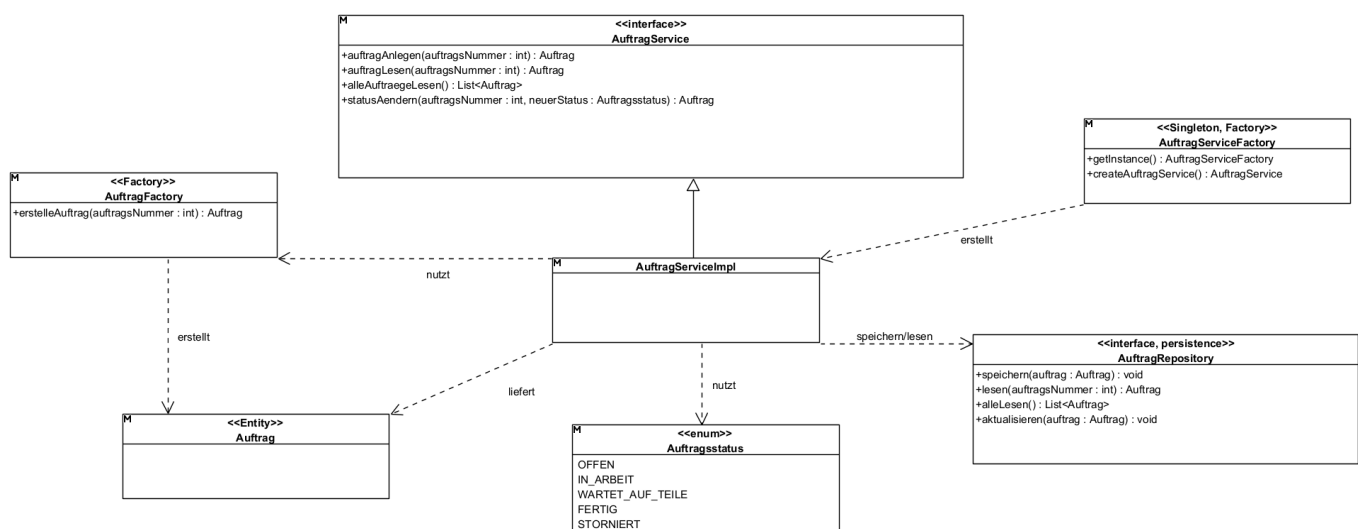


Abbildung 4: UML Klassendiagramm der Business-Schicht

Pattern	Klasse / Schnittstelle	Begründung
Factory	AuftragFactory	Erzeugt Auftrag-Objekte zentral, damit die Objekterstellung nicht im ConsoleClient verteilt wird.
Factory	AuftragServiceFactory	Erzeugt bzw. liefert den Business-Service zentral.
Singleton	AuftragServiceFactory	Die Service-Erzeugung wird zentral gehalten und nicht mehrfach im Code verteilt.
Repository	AuftragRepository	Kapselt den Datenzugriff für Auftrag-Objekte hinter einer Schnittstelle.

4.6 Designmodell Persistence-Schicht

Die Persistence-Schicht kapselt das Speichern und Lesen von Auftrag-Objekten. Für die erste Iteration wird keine echte Datenbank verwendet. AuftragRepositoryMock speichert die Aufträge im Hauptspeicher mit einer Java Collection.

Methode	Rückgabe	Parameter	Zweck
speichern	void	auftrag: Auftrag	Speichert einen neuen Auftrag in der Mock-DB.
lesen	Auftrag	auftragsNummer: int	Liest einen einzelnen Auftrag anhand der Auftragsnummer.
alleLesen	List<Auftrag>	-	Liest alle gespeicherten Aufträge.
aktualisieren	void	auftrag: Auftrag	Aktualisiert einen bestehenden Auftrag, zum Beispiel nach Statuswechsel.

UML Klassendiagramm: Persistence-Schicht

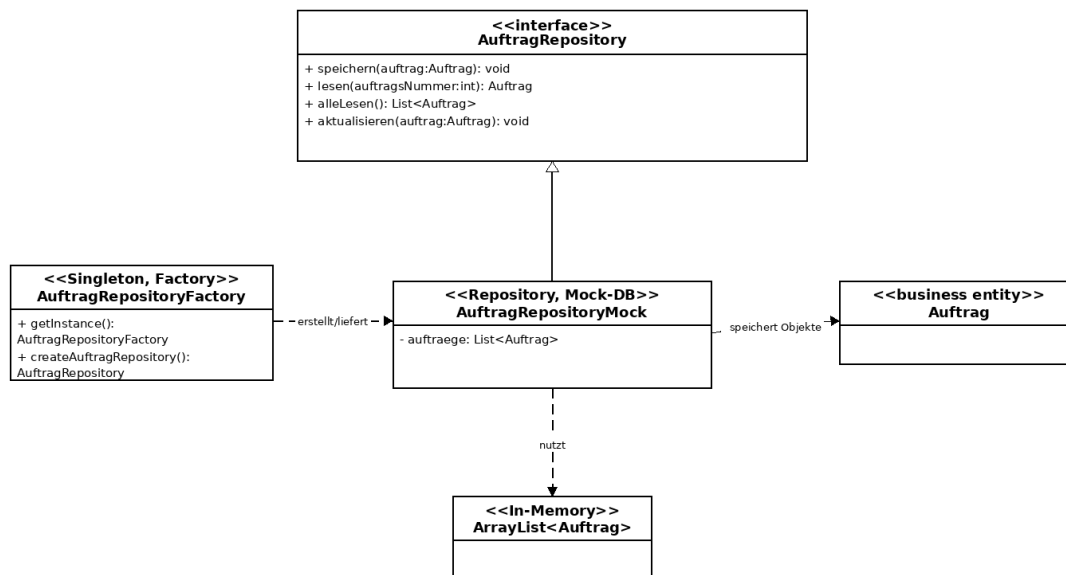


Abbildung 5: UML Klassendiagramm der Persistence-Schicht

Nicht verwendet	Stattdessen verwendet
MariaDB	AuftragRepositoryMock
JDBC-Verbindung	Java Collection im Hauptspeicher
SQL-Tabellen	Objekte vom Typ Auftrag
Dateisystem-Speicherung	In-Memory Mock-DB

4.7 Integriertes Designmodell

Das integrierte Designmodell zeigt, wie Presentation, Business und Persistence zusammenspielen. Die Presentation-Schicht kennt nur die Business-Schicht. Die Business-Schicht koordiniert den Use Case und greift über Repository-Schnittstellen auf die Persistence-Schicht zu.

4.7.1 UML Komponentendiagramm

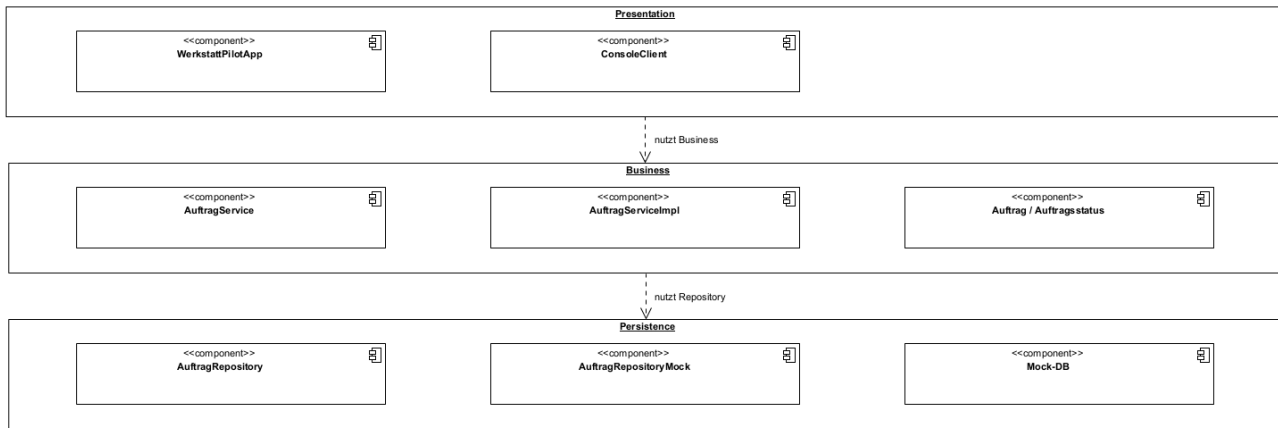


Abbildung 6: Komponentendiagramm mit 3-Schichtenarchitektur

4.7.2 UML Klassendiagramm integriert

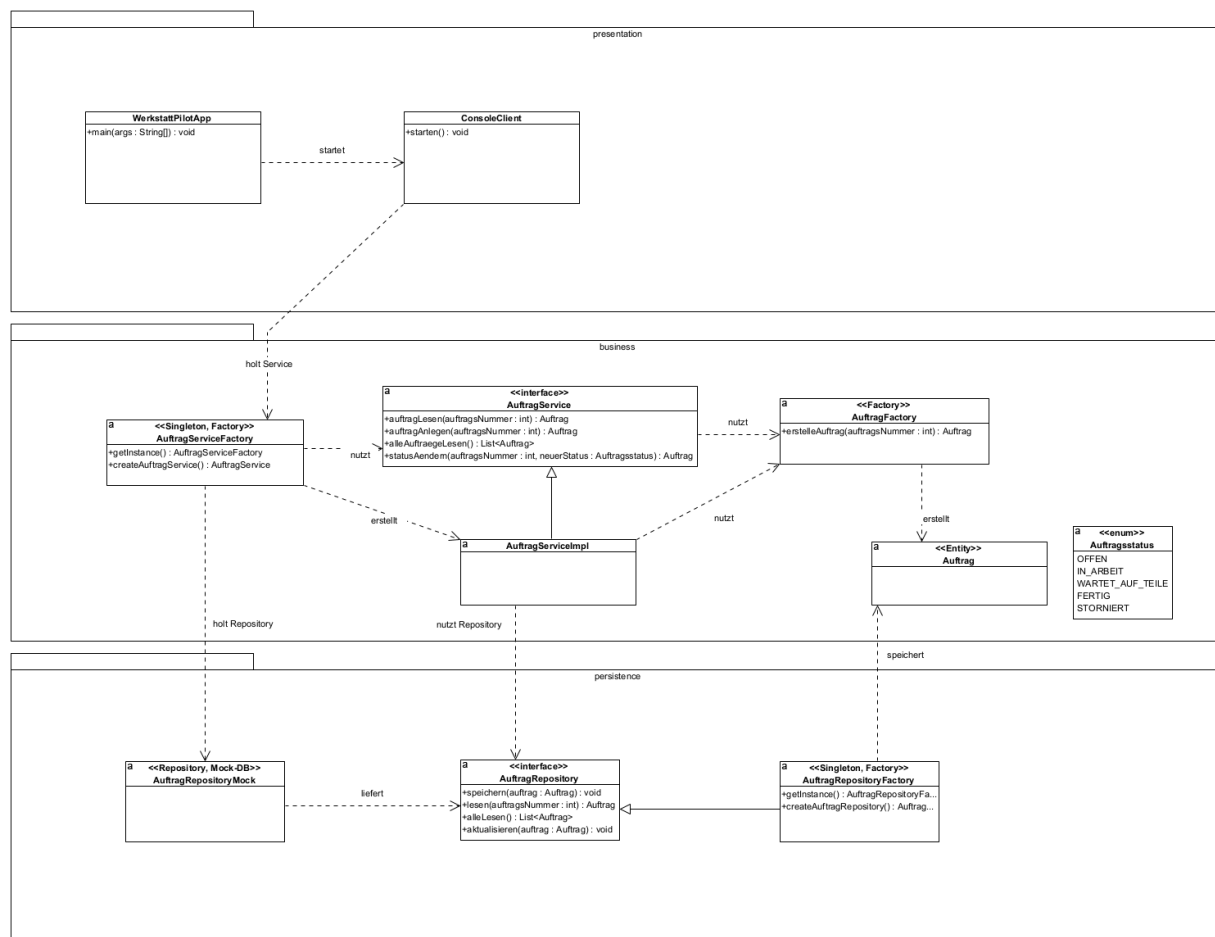


Abbildung 7: integriertes Klassendiagramm

4.7.3 Vollständigkeitsprüfung des integrierten Modells

Prüfrage	Ergebnis
Kann die Anwendung gestartet werden?	Ja. WerkstattPilotApp verwendet ConsoleClient.
Kann UC1 Auftrag anlegen ausgeführt werden?	Ja. ConsoleClient ruft AuftragService auf. AuftragServiceImpl erstellt und speichert einen Auftrag.
Kann UC2 Auftragsübersicht und Statuswechsel ausgeführt werden?	Ja. AuftragService bietet Lesen aller Aufträge und statusAendern an.
Ist die Datenhaltung gekapselt?	Ja. Zugriff erfolgt über AuftragRepository. Die konkrete Mock-Implementierung bleibt austauschbar.
Sind die Patterns sichtbar?	Ja. Factory, Singleton und Repository sind im Modell gekennzeichnet.
Ist das Modell konsistent mit Aufgabe 2?	Ja. Packages und Ordner entsprechen presentation, business und persistence. MariaDB ist entfernt.

4.8 Sequenzdiagramm UC1 Auftrag anlegen

Das Sequenzdiagramm überprüft den Normalablauf des Use Case UC1 Auftrag anlegen. Es zeigt, dass ConsoleClient, AuftragService, AuftragFactory, AuftragRepository und Mock-DB sauber zusammenarbeiten.

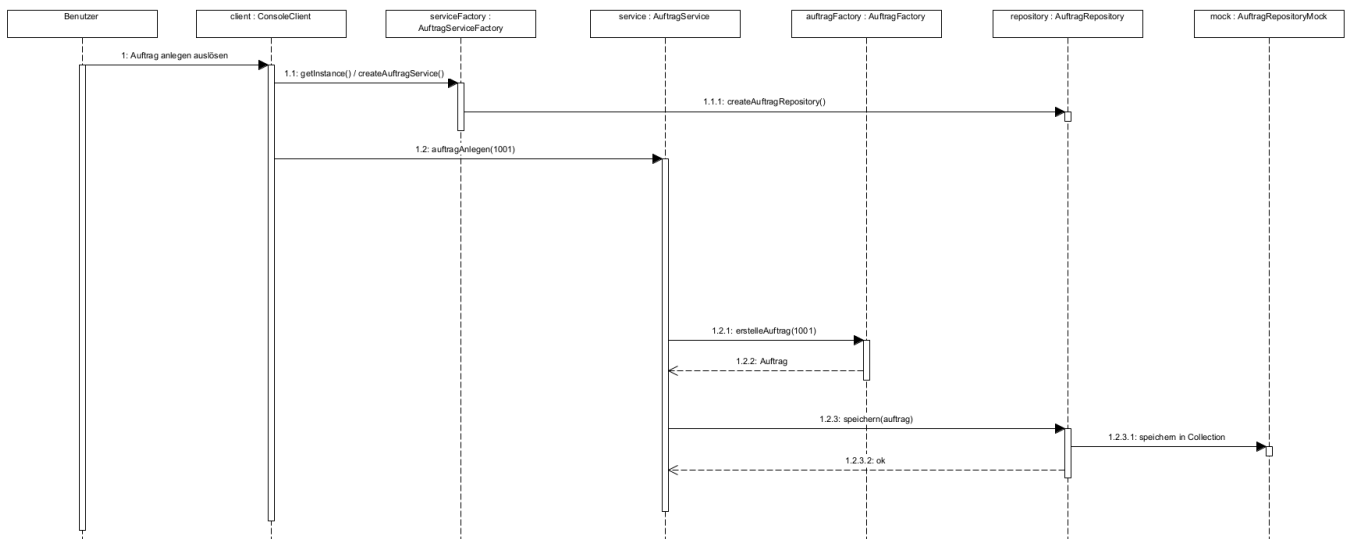


Abbildung 8: Sequenzdiagramm für UC1 Auftrag anlegen

Prüfpunkt	Ergebnis
Fachliche Koordination	AuftragService koordiniert die Erstellung und Speicherung des Auftrags.
Objekterstellung	AuftragFactory erzeugt das fachliche Auftrag-Objekt zentral.
Speicherung	AuftragRepository kapselt den Datenzugriff. AuftragRepositoryMock speichert in der Mock-DB.
Konsistenz	Der Ablauf benötigt keine MariaDB und passt zur Package-Struktur.

4.9 Glossar der Design-Verantwortlichkeiten

Das Glossar beschreibt die Verantwortlichkeiten jeder Klasse und Schnittstelle im Designmodell. Es erklärt nicht die UML-Notation, sondern den konkreten Zweck im Projekt WerkstattPilot.

Element	Schicht	Typ	Verantwortlichkeit
WerkstattPilotApp	Presentation	Klasse	Startet die Anwendung und verwendet ein ConsoleClient-Objekt für den Endbenutzertest.
ConsoleClient	Presentation	Klasse	Erstellt Testdaten, ruft die Business-Schicht auf und gibt Aufträge auf der Konsole aus.
AuftragService	Business	Schnittstelle	Definiert die fachlichen Operationen für Auftrag anlegen, Auftrag lesen, alle Aufträge lesen und Status ändern.
AuftragServiceImpl	Business	Klasse	Setzt die fachlichen Operationen um und koordiniert Factory und Repository.
AuftragServiceFactory	Business	Singleton / Factory	Liefert zentral eine AuftragService-Instanz und kapselt die Service-Erzeugung.
AuftragFactory	Business	Factory	Erzeugt Auftrag-Objekte an einer zentralen Stelle.
Auftrag	Business	Fachliche Klasse	Repräsentiert den Werkstattauftrag der ersten Iteration und wird im Prototyp auf auftragsNummer und status reduziert.
Auftragsstatus	Business	Enum / Wertetyp	Definiert die erlaubten Statuswerte OFFEN, IN_ARBEIT, WARTET_AUF_TEILE, FERTIG und STORNIERT.
AuftragRepository	Persistence	Schnittstelle / Repository	Definiert die Datenzugriffsoperationen für Speichern, Lesen, alle Lesen und Aktualisieren von Aufträgen.
AuftragRepositoryMock	Persistence	Repository-Implementierung	Speichert Auftrag-Objekte im Hauptspeicher und ersetzt eine echte Datenbank.
AuftragRepositoryFactory	Persistence	Singleton / Factory	Liefert zentral eine Repository-Instanz und kapselt die Wahl der konkreten Persistence-Implementierung.

4.10 Rückmeldungen des Dozenten

Datum	Rückmeldung	Umsetzung / Einarbeitung	Status
Di, 21.04.2026	Der Dozent hat bestätigt, dass das Projekt auf einem guten Weg ist.	Das Designmodell bleibt auf die erste Iteration fokussiert und wird nicht unnötig erweitert.	berücksichtigt
Di, 05.05.2026	Im Grossen und Ganzen gut. Einzelne kleine Anpassungen sind noch offen, insbesondere bei Konsistenzen.	MariaDB wurde aus dem Design entfernt. Die Persistence-Schicht ist nun konsistent als Mock-DB / In-Memory Repository modelliert. Packages und Ordnerstruktur wurden abgeglichen.	Noch offen

4.11 Arbeitsprotokoll und Quellen

Die detaillierten Arbeitseinträge zu Aufgabe 4 sind im finalen Arbeitsprotokoll enthalten.

Quelle	Verwendung im Projekt
Aufgabenblatt Aufgabe 4	Vorgaben für Business-Schicht, Persistence-Schicht, integriertes Designmodell, Komponentendiagramm, Sequenzdiagramm, Patterns und Glossar.
Aufgabe 2 und Aufgabe 3	Grundlage für Ordnerstruktur, Packages, Mock-DB-Entscheid und zentrale fachliche Klasse Auftrag.
Aufgabenblatt Aufgabe 5	Vorgaben für spätere Implementation mit Mock-DB, Unit-Tests, ConsoleClient und Javadoc.

5 Aufgabe 5: Objektorientierte Implementation / Programmierung (OOP)

Java-Prototyp für die fachliche Klasse Auftrag

Java-Prototyp, Tests, Javadoc und Endbenutzertest

5.1 Prüfungsabdeckung Aufgabe 5

Prüfziel / Bewertungsgegenstand	Abdeckung	Status	Umsetzung in der finalen Abgabe
Persistence-Schicht	Kapitel 5.5	erfüllt	Code kompiliert, Mock-DB mit ArrayList geprüft.
Business-Schicht	Kapitel 5.4	erfüllt	AuftragService, AuftragServiceImpl und Factory geprüft.
Presentation-Schicht	Kapitel 5.6	erfüllt	WerkstattPilotApp und ConsoleClient vorhanden und lauffähig.
Unit-Tests Persistence/Business	Kapitel 5.7	erfüllt	Einfache Java-Testklassen ohne externe Bibliothek erfolgreich ausgeführt.
Endbenutzertest	Kapitel 5.8	erfüllt	Konsolenausgabe mit drei Aufträgen dokumentiert.
Javadoc	Kapitel 5.9	erfüllt	Javadoc erzeugt und im Projektordner javadoc_html abgelegt.
Präsentation	Kapitel 5.10	erfüllt	Präsentationsstruktur vorbereitet und als Abgabedatei erstellt.
Arbeitsprotokoll / Schreibweise	Kapitel 5.12 und Gesamtdokument	erfüllt	Tippfehler und offene Status bereinigt.

5.2 Ziel und Grundlage der Implementation

Dieses Kapitel beschreibt die Umsetzung von Aufgabe 5 für WerkstattPilot. Grundlage ist das Designmodell aus Aufgabe 4. Der Prototyp ist bewusst klein gehalten und beschränkt sich auf den geforderten Umfang der ersten Iteration.

Die erste Iteration fokussiert auf Auftrag anlegen sowie Auftragsübersicht mit Statusänderung. Für Aufgabe 5 wird deshalb die fachliche Klasse Auftrag umgesetzt.

Grundlage	Bedeutung für Aufgabe 5
Aufgabe 2	Projektumgebung, Java-Projekt, Packages und Mock-DB-Entscheid.
Aufgabe 3	Auftrag als zentrale fachliche Klasse und Statusmodell.
Aufgabe 4	Designmodell mit Presentation-, Business- und Persistence-Schicht.
Aufgabe 5	Java-Prototyp, Tests, ConsoleClient, Javadoc und Präsentation.

5.3 Umfang und Annahmen

Punkt	Umsetzung
Gewählte fachliche Klasse	Auftrag
Attribut 1	auftragsNummer: int
Attribut 2	status: Auftragsstatus
Statuswerte	OFFEN, IN_ARBEIT, WARTET_AUF_TEILE, FERTIG, STORNIERT
Speicherung	In-memory Mock-DB mit ArrayList<Auftrag>
Testdaten	hart im Code, keine Benutzereingaben

Nicht umgesetzt	Begründung
Maven / JUnit	Es wird bewusst nur Basic Java ohne externe Bibliotheken verwendet.
Echte Datenbank	Aufgabe 5 verlangt Mock-DB statt DB.
GUI	Presentation-Schicht ist ein ConsoleClient.
Kunde, Fahrzeug, Beanstandung als Klassen	In Aufgabe 5 werden nur zwei Attribute der Klasse Auftrag umgesetzt.

5.4 Projektstruktur und Packages

Das Projekt ist ein einfaches Java-Projekt. Die Struktur entspricht der 3-Schichtenarchitektur aus Aufgabe 2 und Aufgabe 4.

Package	Verantwortung
presentation	Start der Anwendung, ConsoleClient, Konsolenausgabe.
business	Auftrag, Status, Service, Service-Implementierung und Factories.
persistence	Repository-Schnittstelle, Mock-Repository und Repository-Factory.
test/business	Einfache Java-Tests für die Business-Schicht.
test/persistence	Einfache Java-Tests für die Persistence-Schicht.

```

WerkstattPilot/
.project
.classpath
src/main/java/ch/werkstattteam/werkstattpilot/
    presentation/
    business/
    persistence/
src/test/java/ch/werkstattteam/werkstattpilot/test/
    business/
    persistence/
javadoc_html/

```

5.5 Implementation der Business-Schicht

Die Business-Schicht enthält die fachliche Logik für Auftrag anlegen, Auftrag lesen, alle Aufträge lesen und Status ändern.

Klasse / Schnittstelle	Umsetzung im Code
Auftrag	Fachliche Klasse mit genau zwei Attributen: auftragsNummer und status.
Auftragsstatus	Enum mit den erlaubten Statuswerten.
AuftragFactory	Erstellt neue Auftrag-Objekte mit Status OFFEN.
AuftragService	Schnittstelle für die fachlichen Operationen.
AuftragServiceImpl	Konkrete Umsetzung der fachlichen Operationen.
AuftragServiceFactory	Einfache Factory als Singleton für den Service.

Methode	Rückgabe	Parameter	Zweck
auftragAnlegen	Auftrag	int auftragsNummer	Erstellt und speichert einen neuen Auftrag.
auftragLesen	Auftrag	int auftragsNummer	Liest einen Auftrag anhand der Nummer.
alleAuftraegeLesen	List<Auftrag>	-	Liest alle gespeicherten Aufträge.
statusAendern	Auftrag	int auftragsNummer, Auftragsstatus neuerStatus	Ändert den Status eines bestehenden Auftrags.

Regel	Umsetzung
Neuer Auftrag	Ein neuer Auftrag wird über AuftragFactory mit Status OFFEN erstellt.
Auftragsnummer	Die Auftragsnummer muss grösser als 0 sein.
Status	Beim Setzen oder Ändern darf der Status nicht leer sein.
Unbekannter Auftrag	Beim Statuswechsel eines unbekannten Auftrags wird eine IllegalArgumentException geworfen.

5.6 Implementation der Persistence-Schicht

Die Persistence-Schicht speichert und liest Auftrag-Objekte. Es wird keine echte Datenbank verwendet. AuftragRepositoryMock speichert die Daten während der Programmausführung in einer ArrayList im Hauptspeicher.

Klasse / Schnittstelle	Umsetzung im Code
AuftragRepository	Interface für Speichern, Lesen, alle Lesen und Aktualisieren.
AuftragRepositoryMock	Mock-DB mit ArrayList<Auftrag>.
AuftragRepositoryFactory	Einfache Factory als Singleton für das Repository.
speichern(Auftrag auftrag)	Speichert einen neuen Auftrag in der Liste. Doppelte Nummern sind nicht erlaubt.
lesen(int auftragsNummer)	Sucht einen Auftrag anhand der Auftragsnummer.
alleLesen()	Gibt eine Kopie der Liste zurück.
aktualisieren(Auftrag auftrag)	Sucht einen Auftrag in der Liste und ersetzt ihn.

5.7 Implementation der Presentation-Schicht

Die Presentation-Schicht besteht aus WerkstattPilotApp und ConsoleClient. Die Klasse WerkstattPilotApp enthält die main-Methode. Der ConsoleClient führt den Endbenutzertest aus. Es gibt keine Eingaben über die Tastatur.

Klasse	Verantwortung
WerkstattPilotApp	Startklasse mit main-Methode. Erstellt ein ConsoleClient-Objekt und startet dieses.
ConsoleClient	Erstellt Testdaten, ruft den AuftragService auf und gibt die Aufträge auf der Konsole aus.

5.8 Unit-Tests

Die Tests sind einfache Java-Testklassen ohne externe Testbibliothek. Jede Testklasse besitzt eine main-Methode und eine eigene pruefe-Methode. Wenn eine Prüfung fehlschlägt, wird eine RuntimeException geworfen.

Testklasse	Testfälle / Prüfung
AuftragRepositoryTest	speichernUndLesen, alleLesen, aktualisieren, doppelteAuftragsNummer, unbekanntenAuftragAktualisieren
AuftragServiceTest	auftragAnlegen, statusAendern, alleAuftraegeLesen, unbekannteAuftragsNummer, ungeltigeAuftragsNummer, statusDarfNichtLeerSein

5.9 Endbenutzertest

Der Endbenutzertest wird über WerkstattPilotApp gestartet. Der ConsoleClient erstellt drei Aufträge und ändert anschliessend Statuswerte.

Auftragsnummer	Startstatus	Status nach Ablauf
1001	OFFEN	FERTIG
1002	OFFEN	IN_ARBEIT
1003	OFFEN	WARTET_AUF_TEILE

```

Compile: erfolgreich
Persistence-Test: Alle Persistence-Tests erfolgreich.
Business-Test: Alle Business-Tests erfolgreich.
Endbenutzertest:
WerkstattPilot - Endbenutzertest Aufgabe 5
Erfasste Auftraege:
Auftrag 1001 - Status: OFFEN
Auftrag 1002 - Status: IN_ARBEIT
Auftrag 1003 - Status: WARTET_AUF_TEILE

Status von Auftrag 1001 wird auf FERTIG gesetzt.

Auftraege nach Statusaenderung:
Auftrag 1001 - Status: FERTIG
Auftrag 1002 - Status: IN_ARBEIT
Auftrag 1003 - Status: WARTET_AUF_TEILE

Javadoc: erfolgreich erzeugt, Startseite unter javadoc_html/index.html

```

5.10 Javadoc und Code-Dokumentation

Die Java-Dateien enthalten Dateiköpfe und einfache Javadoc-Kommentare. Dokumentiert sind Klassen, Schnittstellen, Factories, wichtige Methoden sowie Getter und Setter. Die HTML-Seiten wurden nach dem finalen Code-Stand erzeugt.

Vorgabe	Umsetzung im Code
Dateiköpfe	Jede Java-Datei enthält Projekt, Aufgabe, Datei und Autoren.
Interfaces	AuftragService und AuftragRepository sind dokumentiert.
Factories	AuftragFactory, AuftragServiceFactory und AuftragRepositoryFactory sind dokumentiert.
Getter / Setter	Getter und Setter in Auftrag sind dokumentiert.
HTML-Dokumentation	Javadoc wurde erzeugt und liegt unter javadoc_html/index.html.

5.10.1 Befehl zum Erzeugen der Javadoc-HTML-Seiten

```

javadoc -encoding UTF-8 -charset UTF-8 -docencoding UTF-8 -d javadoc_html -sourcepath
src/main/java -subpackages ch.werkstattteam.werkstattpilot

```

5.11 Vorbereitung der Präsentation

Teil	Inhalt
Projektumgebung	Java-Projekt, 3-Schichtenstruktur und Mock-DB erklären.
Analyse / Design	Auftrag als zentrale Klasse und Repository/Factory kurz erklären.
Prototyp	WerkstattPilotApp starten und Konsolenausgabe zeigen.
Tests / Javadoc	Testklassen starten und Javadoc-HTML zeigen.

5.12 Rückmeldungen des Dozenten

Datum	Rückmeldung	Umsetzung / Einarbeitung	Status
Di, 21.04.2026	Wir sind auf dem guten Weg.	Projektumfang nicht unnötig erweitert.	berücksichtigt
Di, 05.05.2026	Im Grossen und Ganzen gut. Einzelne Konsistenzen noch offen.	MariaDB ausgebaut; nur Java und Mock-DB umgesetzt.	umgesetzt

5.13 Arbeitsprotokoll und Quellen

Die detaillierten Arbeitseinträge zu Aufgabe 5 sind im finalen Arbeitsprotokoll enthalten.

Quelle	Verwendung im Projekt
Aufgabenblatt Aufgabe 5	Vorgaben für Java-Prototyp, Tests, ConsoleClient und Javadoc.
Aufgabe 2 - Projektumgebung	Grundlage für Java, Packages und Mock-DB.
Aufgabe 3 - Analysemodell	Grundlage für Auftrag und Status.
Aufgabe 4 - Designmodell	Grundlage für 3-Schichtenstruktur, Service, Repository und Factory.
Java-Dokumentation / JDK	Grundlage für Java-Code und Javadoc-Erzeugung.

6 Transparenzhinweis KI

6.1 Allgemeiner Umgang mit Künstlicher Intelligenz

Im Sinne der Transparenz und der wissenschaftlichen Redlichkeit legen wir hiermit offen, dass im Rahmen des Projekts WerkstattPilot – umfassend die Projektphasen von der Initialisierung über die Analyse und das Design bis hin zur fertigen Implementierung – generative Künstliche Intelligenz (KI) als unterstützendes Werkzeug eingesetzt wurde.

Der Einsatz erfolgte gezielt zur Steigerung der redaktionellen Qualität und der visuellen Präsentation, nicht jedoch zur Erzeugung inhaltlicher oder technischer Kernkonzepte.

6.2 Konkrete Einsatzbereiche der KI

Die Unterstützung durch KI-Systeme beschränkte sich im Wesentlichen auf die folgenden drei Bereiche:

- **Textglättung und stilistische Optimierung**
Die fachlichen Formulierungen, Begründungen und Architekturentscheidungen wurden vollständig von den Autorinnen und Autoren erarbeitet.
Eine KI wurde anschliessend als digitaler Lektor genutzt, um die Rohformulierungen in einen durchgehend präzisen, formalen und einheitlichen technischen Dokumentationsstil zu überführen sowie Rechtschreib- und Grammatikprüfungen durchzuführen.
- **Visuelle Darstellung und Tabellenstrukturen**
Zur übersichtlichen Gliederung der Dokumente, zur Erstellung strukturierter Methodengegenüberstellungen sowie zur Formatierung der verschiedenen Prüfmatrizen wurden KI-gestützte Strukturierungsvorschläge herangezogen, um die Lesbarkeit der Berichte zu maximieren.
- **Generierung des Titelbilds (Deckblatt)** Das einheitlich auf allen Dokumenten verwendete Projekt- und Profilbild (Symbolbild für die digitale Auftragsabwicklung in einer Kfz-Werkstatt) wurde mithilfe eines generativen Bildmodells erstellt.

6.3 Geistige Eigenleistung und inhaltliche Verantwortung

Alle wesentlichen Projektarbeiten und intellektuellen Kernleistungen wurden vollständig eigenständig durch die Arbeitsgruppe erbracht.

Dazu gehören insbesondere:

- Die Definition des Projektumfangs sowie die Erhebung und Ausarbeitung der fachlichen Anforderungen.
- Die softwarearchitektonischen Entscheidungen (z.B. Drei-Schichten-Architektur, In-Memory-Datenhaltung via Mock-DB).
- Das Design und die Definition der UML-Modelle (Fachklassenmodell, Zustandsmodell, integrierte Klassendiagramme und Sequenzdiagramme).
- Die vollständige Programmierung des Java-Prototyps (Klassen, Interfaces, Singletons und Factories) einschliesslich der zugehörigen Unit-Tests.